

La Classe Fantôme : évaluer un contenu pédagogique généré par sa rétention dans une classe synthétique

Artus Cescon Aymeric Hiegel Moustapha Oumeziane Arthur Gallier Estebane Brouillard

Hackathon EPITA : IA agentique au service de l'éducation

Septembre 2026

1 Résumé

Générer un contenu pédagogique avec un modèle de langage est devenu bon marché. L'évaluer ne l'est pas. Le patron générer, critiquer, réviser [13, 14] juge un contenu à l'instant où il est produit. Or la propriété qui intéresse un enseignant, ce qu'il en reste, est temporelle. Nous décrivons La Classe Fantôme, un système multi-agents qui note un contenu généré sur sa rétention dans une classe synthétique. À partir d'un objectif donné en une phrase, six rôles produisent cinq séances. Cinq élèves-agents dotés d'une mémoire longitudinale et d'une méprise nommée suivent ces séances. Les élèves répondent aux exercices, dont la réponse attendue leur a été retirée. Leurs réponses sont corrigées, ce qui donne un taux de réussite par concept, recalculé à chaque séance où le concept est retesté. On en déduit une demi-vie par concept [4]. Deux propriétés structurent l'architecture. Le signal de qualité du contenu et le signal de plausibilité de la simulation circulent dans des canaux disjoints. Le contrôle de plausibilité, le DriftWatcher, trois règles déterministes, est le seul composant du système à ne faire aucun appel au modèle. Sur un run de démonstration de 60 appels, le concept le plus fragile perd 25 points de réussite en trois séances. Quarante points séparent le profil le plus solide du plus fragile, et la réécriture rapporte 24 points en moyenne sur le concept visé. Ces chiffres illustrent le fonctionnement du système. Ils ne constituent pas un résultat expérimental, et la section 9 dit ce qu'ils ne prouvent pas.

2 Introduction

2.1 Le problème d'évaluation

Générer un cours avec un modèle de langage coûte aujourd'hui très peu. En quelques secondes, un modèle produit une leçon structurée, des exercices calibrés et une progression cohérente. Ce coût quasi nul déplace la difficulté vers l'aval. La question n'est plus de produire un contenu pédagogique, mais de savoir s'il vaut quelque chose. Cette évaluation, elle, reste coûteuse.

Le patron dominant pour y répondre consiste à faire relire le contenu, par un humain ou par un second modèle, puis à corriger ce qui est signalé. Ce schéma générer, critiquer, réviser est documenté et adopté [13, 14]. Il porte cependant un angle mort de construction : il évalue le contenu au moment où il est produit, sur la base d'une lecture unique. Un cours peut être jugé clair et validé sans réserve par son relecteur, et ne rien laisser derrière lui trois séances plus tard.

Faire porter la relecture par un second modèle ne referme pas cet angle mort. Les modèles de langage ne savent pas encore se corriger de façon fiable par leur seul jugement [15]. Un évaluateur automatique tend en outre à reconnaître et à favoriser les productions qui ressemblent aux siennes [12]. La relecture automatisée hérite donc des mêmes limites que la relecture humaine instantanée, en plus d'un biais qui lui est propre.

La propriété qui intéresse un enseignant, la rétention, est de nature temporelle : elle se mesure dans la durée, pas à l'instant de la lecture. La courbe de l'oubli décrit depuis plus d'un siècle une décroissance de la mémorisation dans le temps [1]. L'effet de test montre que se soumettre à une évaluation modifie cette décroissance [2], et la pratique espacée en confirme l'effet à travers des dizaines d'études [3]. Aucune relecture instantanée, humaine ou automatisée, ne peut mesurer une propriété qui ne se révèle que plusieurs séances après la lecture.

2.2 Thèse : noter un contenu sur ce qu'il en reste

La Classe Fantôme part de ce constat pour poser une thèse inverse de celle du patron générer, critiquer, réviser : noter un contenu non pas sur l'effet qu'il produit à l'instant où il est lu, mais sur ce qu'il en reste plusieurs séances plus tard.

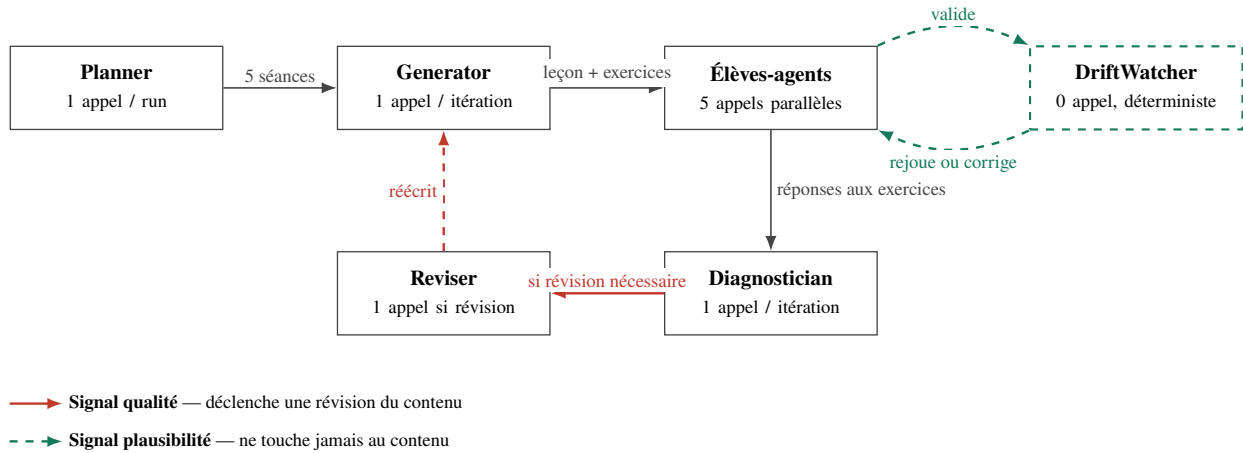


Figure 1: Les six rôles et les deux canaux de signal. Le DriftWatcher est dessiné en pointillés parce qu’il est le seul composant du système à ne faire aucun appel au modèle de langage. Aucune flèche ne relie le canal plausibilité au contenu pédagogique : c’est la contrainte structurelle que l’architecture cherche à garantir.

Le dispositif traduit cette thèse. Un enseignant donne un objectif pédagogique en une phrase. Le système génère cinq séances, que suivent cinq élèves-agents dotés chacun d’une mémoire longitudinale. Ils répondent aux exercices, leur mémoire évolue d’une séance à l’autre, et les concepts déjà enseignés leur sont reproposés plusieurs séances après leur introduction. La métrique centrale n’est donc pas une impression de qualité recueillie une fois, mais le taux de réussite par concept, recalculé à chaque séance où ce concept est retesté. Cette mesure répétée donne une courbe et non un point. Elle permet d’en déduire une demi-vie par concept, au sens où l’entend la régression de demi-vie appliquée à l’apprentissage [4].

Des élèves simulés par modèle de langage existent déjà dans la littérature, mais avec un objet différent. Ils servent à former des enseignants [8], à simuler des comportements humains dans des environnements ouverts [7], ou à répliquer des études menées sur des sujets humains [9], sans que le contenu pédagogique lui-même soit noté sur sa rétention. La modélisation de l’acquisition dans le temps constitue par ailleurs un champ établi, du knowledge tracing [5] à ses variantes profondes [6]. La Classe Fantôme s’en distingue par son objet : ces travaux cherchent à prédire la performance d’un apprenant réel, quand ce rapport cherche à comparer des contenus entre eux à partir d’une rétention simulée.

2.3 Contributions et périmètre

Ce rapport apporte quatre éléments. D’abord, une architecture à six rôles dans laquelle deux canaux de signal, le canal qualité et le canal plausibilité, restent disjoints : le premier déclenche une réécriture du contenu, le second ne touche jamais au contenu. Ensuite, un contrôle de plausibilité déterministe, le DriftWatcher, seul composant du système qui ne repose sur aucun appel au modèle. Puis une métrique de rétention par concept et la demi-vie qui s’en déduit. Ces deux mesures sont construites pour capter ce qu’un contenu laisse derrière lui plutôt que l’impression qu’il produit à la lecture. Enfin, un run de démonstration instrumenté de bout en bout, qui donne à voir le système en fonctionnement sur chacun de ces éléments.

Le statut de ces chiffres doit être posé sans détour. Ils proviennent d’un run de démonstration construit pour exercer tous les cas de figure à la fois, et détaillé en sous-section 6.3. Ils illustrent le fonctionnement du système et ne constituent pas un résultat expérimental. Ce que ce rapport ne prouve pas est explicité en section 9.

3 Architecture du système

Cette section détaille l’architecture qui met en œuvre le cycle génération, simulation et diagnostic : les six rôles qui y participent, la séparation en deux canaux de signal qui la structure, et le déroulé complet d’une séance. La figure 1 en donne la vue d’ensemble. Le tableau 1 en précise le coût en appels au modèle. La figure 2 en détaille la chronologie.

L’orchestration ne s’appuie sur aucun framework d’agents, ni LangChain ni LangGraph. Elle est écrite à la main dans `src/orchestrator.py`, où 108 lignes contiennent toute la boucle. Ce choix est délibéré : sur un projet où l’architecture est l’objet de l’étude, une boucle explicite se lit et se défend intégralement. Un graphe déclaré dans une bibliothèque tierce déplacerait une partie de la logique hors du code qu’on présente.

3.1 Six rôles, un seul appel par responsabilité

Six rôles participent à un run, chacun responsable d’une seule tâche. Le Planner découpe l’objectif fourni par l’enseignant en cinq séances. Il intervient une fois par run, et le plan de séances qu’il produit est soumis à la validation de l’enseignant avant toute génération de contenu. Le Generator rédige ensuite la leçon et les exercices de chaque séance. Chaque exercice

Tableau 1: Les six rôles du système et leur consommation d’appels au modèle. Les comptes de la dernière colonne sont ceux du run de démonstration (5 séances, 8 itérations, 3 révisions).

Rôle	Appels	Run
Planner	1 par run	1
Generator	1 par itération	8
Élève-agent (×5)	1 par élève et par itération	40
Diagnostician	1 par itération	8
Reviser	1 par révision	3
DriftWatcher	aucun	0
Total		60

porte un concept nommé et une réponse attendue, ce qui rend possible la correction automatique en aval. Cinq élèves-agents répondent à ces exercices, réagissent au contenu et réécrivent leur propre mémoire.

Le Diagnostician corrige les réponses reçues, calcule le taux de réussite par concept et diagnostique confusion, ennui ou décrochage. Le Reviser, sollicité seulement quand une révision est nécessaire, traduit ce diagnostic en consignes de réécriture adressées au Generator. Le DriftWatcher, enfin, ne produit aucun contenu pédagogique : il vérifie que la simulation reste crédible, selon des règles déterministes.

Le tableau 1 rend ce partage du travail visible en appels au modèle. Sur les 60 appels du run de démonstration, les élèves-agents en concentrent 40. Ce nombre reflète leur effectif, cinq agents par itération, plutôt qu’une complexité de tâche particulière. Le DriftWatcher, à l’inverse, en consomme 0 : il valide chaque réaction produite par les élèves-agents, mais par des règles déterministes plutôt que par un appel au modèle. C’est ce qui explique qu’il apparaisse en pointillés dans la figure 1.

3.2 Deux canaux de signal qui ne se mélangent jamais

Quand une classe échoue à un exercice, deux explications concurrentes sont possibles : le contenu enseigné est mauvais, ou la simulation qui joue les élèves est invraisemblable. Un système qui confond ces deux explications optimise dans le vide. Il réécrit un cours par ailleurs correct parce qu’un élève-agent a produit une réaction incohérente, ou il laisse filer une dérive de simulation faute de tout mécanisme pour la détecter. La Classe Fantôme sépare les deux signaux dans deux canaux qui ne communiquent jamais : le canal qualité et le canal plausibilité.

Le canal qualité relie le Diagnostician au Reviser puis au Generator. Il mesure la qualité du contenu pédagogique : un taux de réussite qui s’effondre, une confusion collective sur un concept, un ennui manifeste. Un signal sur ce canal déclenche une réécriture du contenu, et rien d’autre. Le canal plausibilité, porté par le DriftWatcher, mesure une chose différente : la crédibilité de la simulation elle-même. Un signal sur ce canal peut invalider la réaction d’un élève-agent et la faire rejouer, ou la corriger de force si le jeu échoue à son tour. Il peut aussi remonter un avertissement de trajectoire au niveau du run entier.

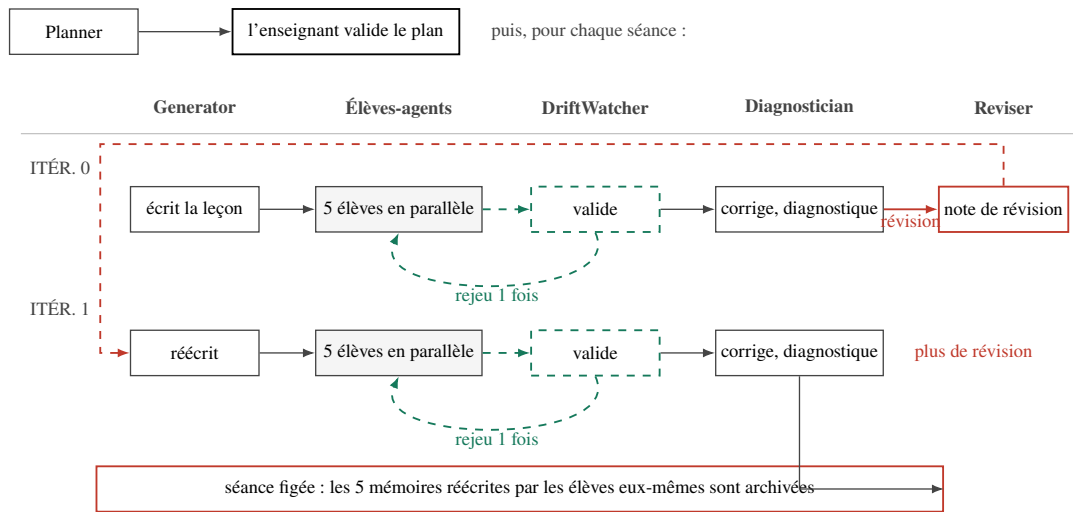
L’asymétrie entre les deux canaux est structurelle plutôt qu’accidentelle : le canal plausibilité dispose d’un chemin vers la mémoire des élèves-agents, mais d’aucun chemin vers le contenu pédagogique. La figure 1 le montre : aucune flèche verte, qui matérialise le canal plausibilité, n’atteint le Generator. Cette séparation évite le piège naturel de ce genre d’architecture, où l’on finirait par réécrire un cours parce que l’élève simulé a déraillé, alors que le contenu n’y est pour rien.

3.3 Déroulé d’une séance et boucle de révision

Le déroulé d’une itération, détaillé dans la figure 2, suit un ordre fixe. Le Generator écrit ou réécrit la leçon et les exercices de la séance. Les cinq élèves-agents y répondent en parallèle. Le DriftWatcher valide chacune de leurs réactions avant que le Diagnostician ne les exploite, jouant ou corrigeant de force celles qui violent une règle. Le Diagnostician corrige alors les réponses et calcule le taux de réussite par concept. Si son diagnostic conclut à une révision nécessaire, le Reviser produit une note de révision et une nouvelle itération démarre sur la même séance. Sinon la séance est figée, et les cinq mémoires réécrites par les élèves-agents eux-mêmes sont archivées. Ce patron générer, critiquer, réviser est connu [13, 14] ; il est appliqué ici au contenu pédagogique plutôt qu’à du texte libre, avec une borne d’itérations explicite.

Le nombre d’itérations par séance est borné à deux ($\text{MAX_ITER} = 2$). Cette borne garantit que le système ne peut pas boucler indéfiniment sur une séance récalcitrante : au-delà de deux itérations, la séance est figée quoi qu’il arrive, même si le Diagnostician continue de signaler un problème.

La parallélisation des cinq élèves-agents d’une même itération, exécutés dans un `ThreadPoolExecutor`, a une conséquence sur le coût du run. Les 60 appels du run de démonstration ne se traduisent qu’en 28 attentes successives sur le chemin critique, puisque les cinq élèves-agents d’une itération sont indépendants entre eux. La section 8.3 chiffre ce point plus en détail.



La boucle s'arrête dès que le Diagnostician ne demande plus de révision, et au plus tard après deux itérations (MAX_ITER = 2).

Figure 2: Déroulé d'une séance. Les cinq élèves d'une même itération partent en parallèle ; la boucle s'arrête dès que le Diagnostician ne demande plus de révision, et au plus tard après deux itérations. C'est ce déroulé qui explique pourquoi un run de 60 appels ne coûte que 28 attentes successives.

Tableau 2: Les cinq personas du run. Chaque profil porte une méprise nommée, et non un trait de caractère : c'est elle qui rend les erreurs reproductibles et les notes du Reviser actionnables.

Identifiant	Méprise caractéristique
eleve_rapide	pense qu'optimiser trop tôt est toujours mieux
eleve_difficulte	confond affectation (=) et égalité (==)
eleve_distrait	oublie les cas limites (listes vides, zéro)
eleve_moyen	pense qu'une boucle for ne peut pas décrémenter
eleve_anxieux	évite de tester son code par peur de se tromper

4 Les élèves-agents

Un élève simulé par un grand modèle de langage est une caricature : il ne se comporte pas comme un adolescent, il reproduit la représentation qu'un modèle se fait d'un adolescent. Cette limite est structurelle. Elle est reprise en détail en section 9.1 et n'est pas contournée ici. Ce que ce projet fait, en revanche, c'est rendre cette caricature exploitable comme instrument de mesure, en donnant à chaque persona une contrainte précise plutôt qu'une humeur diffuse. La démarche s'inscrit dans la lignée des agents génératifs à mémoire [7] et rejoint GPTeach [8], où des élèves simulés par LLM servent déjà à former des enseignants. Les sections qui suivent précisent la nature de cette contrainte, la manière dont elle reste étanche à la réponse attendue, et le traitement de l'oubli qui en découle.

4.1 Des méprises nommées plutôt que des humeurs

Attribuer une humeur à un élève simulé, anxieux, distrait, rapide, ne suffit pas à produire un comportement mesurable. Un adjectif ne dicte aucune erreur précise, et un modèle de langage peut l'incarner de cent façons différentes d'une séance à l'autre. Chacune des cinq personas du run (tableau 2) porte à la place une méprise nommée, une règle fautive et précise que l'élève applique systématiquement. `eleve_difficulte` confond l'affectation et l'égalité, `eleve_moyen` pense qu'une boucle `for` ne peut pas décrémenter, `eleve_anxieux` évite d'exécuter son code par peur de se tromper. Une méprise nommée produit des erreurs reproductibles, qui se retrouvent d'une itération à l'autre dans les réponses de l'élève-agent. Un trait de caractère, lui, se dilue au fil des séances et ne laisse rien de vérifiable dans le contenu produit.

Cette précision se répercute sur la boucle de révision. Quand le Diagnostician relève que la classe échoue sur les boucles `for` à pas négatif, l'erreur caractéristique de `eleve_moyen`, le Reviser ne demande pas au Generator d'être « plus clair ». Il demande d'ajouter un exemple à pas négatif, une consigne actionnable parce que la méprise qui l'a motivée est nommée. Une humeur ne permettrait pas cette précision, puisqu'elle ne pointe vers aucune correction de contenu identifiable. C'est cette correspondance entre méprise et consigne de réécriture qui fait tenir le canal qualité décrit en section 3.2.

Tableau 3: Un exercice de la séance 1, tel qu’il est produit par le Generator et tel qu’il parvient à l’élève-agent. Le champ `expected_answer` est retiré de la charge utile envoyée à l’élève (`src/agents/student.py`) ; sans ce filtrage, l’élève recopierait la solution et tous les taux de réussite de ce rapport vaudraient 100%.

Champ	Valeur
<i>Reçu par l’élève-agent</i>	
question	Après $a = 3$ puis $b = a$ puis $a = 5$, que vaut b ?
concept	variables
<i>Retiré avant l’envoi</i>	
expected_answer	3

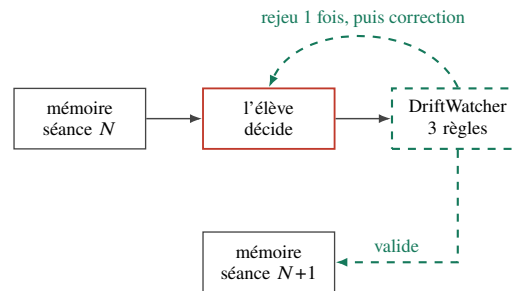


Figure 3: Le cycle de la mémoire. L’oubli n’est calculé par aucune formule : il est produit par l’élève-agent, puis contrôlé. Une mémoire n’est archivée qu’une fois le contrôle passé.

4.2 La précaution sans laquelle tout le reste s’effondre

Chaque exercice produit par le Generator porte un champ `expected_answer`, parce que le Diagnostician en a besoin pour corriger les réponses et calculer le taux de réussite par concept. Cette même réponse ne doit en aucun cas parvenir à l’élève-agent chargé de répondre à l’exercice. Un élève qui voit la solution ne se trompe jamais, quelle que soit sa persona, ce qui viderait la mesure de tout contenu.

Le filtrage est appliqué dans `src/agents/student.py`, qui ne transmet à l’élève-agent que les champs `question` et `concept` de chaque exercice, à l’exclusion de `expected_answer` (tableau 3). C’est une poignée de lignes de code, mais c’est la condition de validité de tout ce rapport. Sans ce filtrage, l’élève-agent recopierait la solution, tous les taux de réussite vaudraient 100%, et aucune courbe de rétention n’aurait de sens.

4.3 L’oubli comme décision, non comme formule

L’oubli n’est codé sous aucune forme paramétrique. Aucune courbe de décroissance n’est câblée dans le système, et aucun coefficient qui ferait chuter un taux de maîtrise d’un pourcentage fixe entre deux séances n’a été réglé à la main. Rien, dans le code de l’élève-agent, ne calcule combien un profil doit oublier d’une séance à l’autre.

Ce que le système fait à la place : chaque élève-agent reçoit sa mémoire en entrée. Cette mémoire liste les concepts maîtrisés, ceux qui sont fragiles, et ceux qui sont déjà oubliés. L’élève décide lui-même, à partir de cette mémoire et de sa persona, ce qu’il retient à l’issue de la séance. La solution paramétrique aurait consisté à reproduire une courbe connue, celle d’Ebbinghaus [1] par exemple, ajustée séance après séance. Le projet s’en écarte délibérément : si l’oubli suivait une formule choisie d’avance, la mesure porterait sur cette formule et non sur le contenu des séances. Cela viderait la comparaison entre une séance ordinaire et une séance révisée de son intérêt.

Cette liberté a un coût. Rien ne garantit qu’un modèle de langage produise, séance après séance, une mutation de mémoire vraisemblable. Un élève-agent pourrait tout aussi bien déclarer maîtriser un concept qu’il n’a jamais vu, ou perdre d’un coup l’intégralité de sa mémoire. La figure 3 situe ce point de bascule dans le cycle complet, entre la décision de l’élève et l’archivage de sa mémoire. La section 5 décrit le contrôle qui s’y applique.

5 Le DriftWatcher : un agent sans modèle de langage

Le Planner, le Generator, les élèves-agents, le Diagnostician et le Reviser reposent tous sur un modèle de langage. Leurs sorties sont probabilistes, et rejouer un run ne produit jamais exactement le même contenu. Le DriftWatcher rompt avec ce régime. C’est le seul des six rôles qui n’effectue aucun appel au modèle (tableau 1) : ses trois règles sont du code ordinaire, appliquées à chaque mutation de mémoire, et leur résultat est reproductible à l’identique d’une exécution à l’autre.

Ce choix est délibéré. Confier ce contrôle à un second modèle de langage, en lui demandant de juger si une mémoire d’élève est plausible, aurait reproduit le problème qu’il s’agit de résoudre : un modèle qui relit la production d’un autre modèle n’offre pas de garantie de correction supérieure à celle du modèle relu [15]. Le DriftWatcher est le point du système

Tableau 4: Les trois règles appliquées par `validate_reaction` à chaque mutation de mémoire produite par un élève-agent. En cas de violation persistante, `clamp_reaction` corrige la mémoire et l’incident est consigné dans le journal du run.

	Contrôle	Correction
R1	Un concept ne peut être déclaré <i>maîtrisé</i> que s’il a été enseigné, dans la séance en cours ou dans une séance antérieure.	retrait de la liste des concepts maîtrisés
R2	Un concept ne peut être déclaré <i>oublié</i> que s’il était auparavant maîtrisé ou fragile.	retrait de la liste des concepts oubliés
R3	L’engagement ne peut pas varier de plus de 0.4 entre deux séances consécutives.	écrêtage à ± 0.4

Tableau 5: Journal des contrôles sur le run de démonstration. Une règle déclenchée au niveau d’une réaction, et un avertissement de trajectoire au niveau du run. Le second ne déclenche aucune réécriture : il est seulement remonté dans le rapport.

Règle	Élève	Décl.
R1 — maîtrise d’un concept non enseigné	<code>eleve_rapide</code>	1
R2 — oubli d’un concept jamais acquis	—	0
R3 — saut d’engagement > 0.4	—	0

Trajectoire suspecte (niveau run) : `eleve_rapide`, engagement au maximum sur 3 itérations consécutives sans aucune erreur, profil trop lisse pour être plausible.

où l’on cesse de demander qu’on fasse confiance à un modèle. On y affirme, avec des règles vérifiables et un code qui se relit et se teste indépendamment, que trois propriétés précises sont respectées.

5.1 Trois règles déterministes

Les trois règles réunies dans le tableau 4 portent toutes sur la cohérence temporelle de la mémoire d’un élève-agent : ce qu’il déclare savoir à la séance courante doit rester compatible avec ce qu’il déclarait savoir à la séance précédente. Aucune des trois ne lit le contenu pédagogique de l’exercice ou de la leçon. Elles ne regardent que l’état de la mémoire avant et après la mutation. C’est cette propriété qui garantit l’étanchéité du canal plausibilité décrite en section 3.2 : le DriftWatcher ne peut, par construction, ni juger ni modifier le contenu d’une séance, puisqu’il ne le voit jamais.

R1 et R2 encadrent les deux mutations d’état possibles pour un concept. On ne peut se déclarer *maîtrisé* sur un concept qui n’a jamais été enseigné, et on ne peut se déclarer *oublié* sur un concept qu’on n’a jamais possédé, ne serait-ce que sous une forme fragile. R3 borne à 0.4 la variation de l’engagement entre deux séances consécutives, pour une raison différente : l’engagement est une variable continue et non un état discret. Une trajectoire qui grimperait ou s’effondrerait arbitrairement d’une séance à l’autre ne serait pas crédible, puisqu’un modèle livré à lui-même n’a aucune raison intrinsèque de la faire évoluer progressivement.

5.2 Rejeu, correction forcée, et journal

Quand une règle est violée, l’élève-agent n’est pas corrigé d’emblée. Il est resollicité une fois, avec la raison précise du rejet insérée dans son prompt, pour lui donner l’occasion de produire de lui-même une mutation de mémoire conforme. Si cette seconde réponse viole encore une règle, `clamp_reaction` corrige la mémoire de force : retrait d’un concept déclaré maîtrisé à tort, ou écrêtage d’un saut d’engagement excessif. L’incident est alors consigné dans le journal du run.

Le rejeu est borné à un seul essai. Au-delà, laisser le système insister reviendrait à négocier avec le modèle jusqu’à obtenir la réponse souhaitée, ce qui romprait le déterminisme que le DriftWatcher est censé garantir. Sur le run de démonstration, le journal des contrôles (tableau 5) ne recense qu’un seul déclenchement : une violation de R1 sur `eleve_rapide`, une maîtrise déclarée sur un concept qui n’avait pas encore été enseigné. R2 et R3 ne se sont déclenchées à aucune reprise sur ce run.

5.3 Contrôles au niveau du run

Un second niveau de contrôle, `check_drift`, ne travaille pas réaction par réaction mais sur l’historique complet des diagnostics d’un run. Il signale trois trajectoires globales invraisemblables : un concept qui stagne malgré des séances de révision ciblées, un ennui déclaré au maximum et constant d’une séance à l’autre, ou une classe dont le profil resterait parfait sans jamais montrer la moindre friction. Ces signaux ne déclenchent jamais de réécriture, à la différence des trois règles de la section 5.1. Ils remontent comme avertissements dans le rapport.

Sur le run de démonstration, un seul avertissement a été levé : `eleve_rapide` affiche un engagement au maximum sur trois itérations consécutives sans la moindre erreur, un profil trop lisse pour être plausible. Sans ce second niveau de contrôle, comme sans les trois règles qui le précèdent, un élève-agent pourrait déclarer maîtriser un concept jamais abordé. L'ensemble des courbes de rétention présentées dans ce rapport deviendrait alors décoratif.

6 Protocole de mesure

6.1 La métrique centrale : le taux de réussite par concept

Le taux de réussite par concept est la métrique centrale du système. Il est calculé par le Diagnostician, qui compare les réponses données par les élèves-agents aux réponses attendues produites par le Generator pour chaque exercice. Cette mesure porte donc sur des réponses à des exercices réels, et non sur une auto-évaluation déclarative de l'élève-agent sur ce qu'il pense avoir compris.

Le taux de réussite est indexé par concept et non par séance. Un concept introduit en séance 1 reste un objet de mesure dans les séances suivantes, qu'il réapparaisse ou non dans un exercice donné. Chaque fois qu'un concept est retesté, le Diagnostician recalcule son taux de réussite, ce qui produit une courbe et non un point isolé.

Cette dernière propriété porte l'ensemble du projet. Elle seule permet de voir un concept se dégrader entre deux séances, là où une mesure prise une seule fois à la fin d'une séance ne donnerait qu'un instantané. La mémoire auto-rapportée par l'élève-agent constitue un signal narratif utile pour diagnostiquer confusion, ennui ou décrochage : la tendance d'engagement (`engagement_trend`), les listes de concepts maîtrisés ou fragiles. Elle n'est cependant pas la métrique objective ; celle-ci vient uniquement des corrections.

6.2 De la courbe à la demi-vie

La demi-vie mesurée ici est définie de façon directe : pour chaque concept retesté après son pic de réussite, on relève la perte en points de taux de réussite et le nombre de séances écoulées depuis ce pic. Le terme est employé par analogie et non au sens strict. Le protocole ne mesure pas un temps de demi-décroissance ajusté sur un modèle paramétrique ; il mesure une pente observée sur un petit nombre de points.

Une version paramétrique de cette idée existe déjà, par régression de demi-vie [4] : elle estime la demi-vie d'un souvenir pour prédire le moment optimal d'une révision chez un apprenant réel. L'usage fait ici est plus modeste et de nature comparative. Il sert à classer les concepts entre eux à l'intérieur d'un même run, pour dire lequel se dégrade le plus vite, sans prétendre prédire un instant de révision individuel. Cette approche a une limite structurelle : un concept introduit en fin de run n'a pas encore de séance après son pic, et ne peut donc pas être caractérisé.

6.3 Le jeu de données présenté dans ce rapport

Le run présenté dans ce rapport est un run de démonstration, construit pour exercer tous les cas de figure à la fois : trois cycles de révision, une correction de plausibilité, et des concepts retestés assez souvent pour dessiner une vraie courbe de rétention. L'objectif donné au système était « faire comprendre les boucles `for/while` à des débutants en programmation, niveau lycée ». Il produit cinq séances, huit itérations, cinq concepts, et soixante appels au modèle. Ces chiffres illustrent le fonctionnement du système. Ils ne constituent pas un résultat expérimental ; ce point est développé en section 9.

Une précision d'honnêteté s'impose sur ce jeu de données précis : le passage d'un concept à l'état « oublié » y est décidé au niveau de la classe entière et non séparément pour chaque élève, ce qui limite ce que l'on peut lire dans la figure 6. La divergence réelle entre profils, elle, s'observe sur la figure 5, construite directement à partir des corrections individuelles.

7 Résultats du run de démonstration

Cette section présente les résultats du run de démonstration dans l'ordre suivant : la courbe de rétention par concept, la demi-vie qui s'en déduit, puis la question de savoir si les cinq élèves-agents divergent réellement ou reproduisent une seule et même réponse, et enfin ce que la boucle de révision rapporte en termes de gain mesuré.

7.1 Rétention par concept

La figure 4 montre deux profils de courbe distincts. Le concept *variables* monte de 40% en séance 1 à 55% en séance 2, puis s'effrite jusqu'à la fin du run pour atteindre 30% en séance 5. La boucle `for`, introduite en séance 2 à 65%, se consolide à 75% en séance 3, avant de redescendre jusqu'à 55% en séance 5.

Ce qui compte dans cette lecture n'est pas le niveau atteint, mais le sens de la pente. Une courbe qui redescend signale un concept que la classe est en train de perdre, et non un contenu qui aurait déplié à un instant donné. C'est l'information qu'une relecture instantanée, humaine ou automatisée, ne peut pas produire, puisqu'elle ne dispose que d'un seul instant d'observation. Les courbes de la figure 4 ont des longueurs inégales parce qu'un concept n'entre dans le graphique qu'à la séance où il est introduit : *boucle while* n'apparaît qu'en séance 3, *conditions imbriquées* qu'en séance 4, *fonctions* qu'en séance 5. Le tableau qui suit chiffre ce que cette courbe laisse voir.

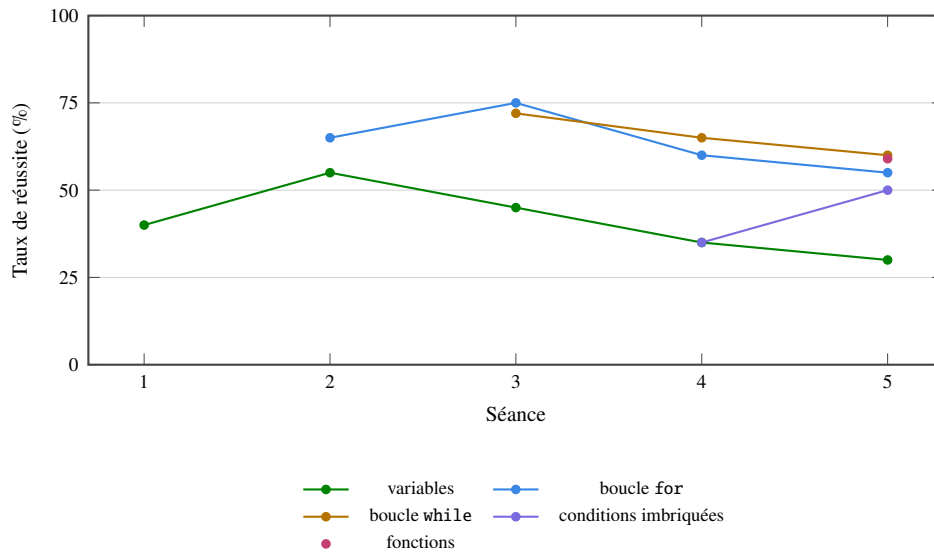


Figure 4: Taux de réussite mesuré à chaque séance où le concept est retesté. Une courbe qui redescend signale un concept que la classe perd, et non un contenu qui a déplu. Un concept n’entre dans le graphique qu’à la séance où il est introduit, ce qui explique les longueurs inégales.

Tableau 6: Demi-vie mesurée. Pour chaque concept retesté après son pic, la perte en points de taux de réussite et le nombre de séances écoulées. Les deux concepts introduits en fin de run (*conditions imbriquées*, *fonctions*) n’ont pas encore de point après leur pic et sont donc absents du tableau.

Concept	Pic	S5	Perte	Séances
variables	55% (S2)	30%	−25 pts	3
boucle for	75% (S3)	55%	−20 pts	2
boucle while	72% (S3)	60%	−12 pts	2

7.2 Demi-vie mesurée

Le tableau 6 traduit ces courbes en une seule valeur par concept. Le concept qui se dégrade le plus vite est *variables* : il perd 25 points de taux de réussite en trois séances après son pic, passant de 55% en séance 2 à 30% en séance 5. La boucle *for* perd 20 points en deux séances, de 75% à 55%, et la boucle *while* perd 12 points en deux séances, de 72% à 60%.

Cette information est le cœur du projet. Elle n’existe que dans la durée, construite à partir de plusieurs séances espacées, et aucun relecteur, humain ou modèle, ne peut la produire en jugeant un cours au moment où il est écrit. Les deux concepts introduits en fin de run, *conditions imbriquées* et *fonctions*, n’ont pas encore de séance après leur pic. Ils n’apparaissent donc pas dans le tableau 6.

7.3 Cinq agents, ou cinq copies du même agent ?

Le système fait jouer cinq personas par le même modèle, ce qui pose une objection sérieuse. Rien ne garantit a priori que ces cinq élèves-agents produisent des réponses distinctes plutôt que de converger vers une seule et même réponse déguisée en cinq voix. Dans ce cas, la classe entière n’aurait aucune valeur d’échantillon, et le run se réduirait à un individu simulé cinq fois. C’est le risque de caricature homogène documenté par [10].

La réponse à cette objection est mesurée sur les corrections du Diagnostician, et non affirmée par construction. La figure 5 montre *eleve_rapide* réussir 80% des exercices sur l’ensemble du run, contre 40% pour *eleve_difficile*. Quarante points séparent le profil le plus solide du plus fragile, et les cinq profils s’échelonnent entre ces deux bornes plutôt que de se regrouper autour d’une seule valeur.

Cet écart est net : la population synthétique se comporte comme une classe et non comme un individu dupliqué cinq fois. Il tient aux méprises nommées de la section 4.1, qui donnent au modèle une contrainte concrète à respecter à chaque itération, là où un simple adjectif de caractère se dilue au fil des séances. La figure 6 montre la trajectoire mémoire séance par séance, avec une réserve qui s’impose : dans ce jeu de données, le passage d’un concept à l’état « oublié » est décidé au niveau de la classe et non par élève. Les cinq élèves-agents n’y suivent donc que deux trajectoires distinctes plutôt que cinq.

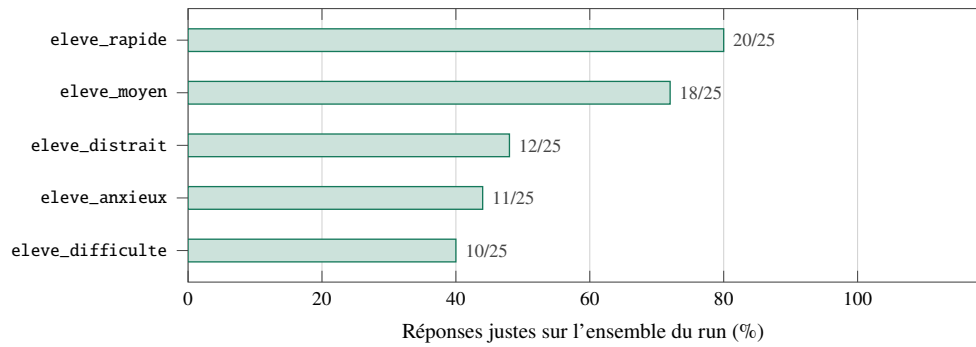


Figure 5: Écart de réussite entre profils sur l'ensemble du run, mesuré sur les corrections du Diagnosticien. Quarante points séparent le profil le plus solide du plus fragile.

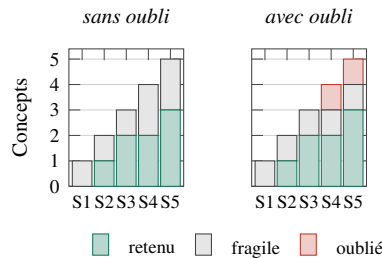


Figure 6: Trajectoire mémoire, séance après séance. Les cinq élèves-agents se répartissent sur deux trajectoires seulement : trois d'entre eux n'oublient jamais rien, tandis que *eleve_difficulte* et *eleve_moyen* perdent le concept *variables* à partir de la séance 4. C'est un effet du jeu de données de démonstration, où le passage d'un concept en « oublié » est décidé au niveau de la classe et non par élève ; la divergence réelle entre profils s'observe sur la figure 5, pas ici.

7.4 Ce que la réécriture rapporte

Le tableau 7 donne le taux de réussite du concept visé avant et après le passage du Reviser, sur les trois séances qui ont déclenché une révision. La réécriture rapporte en moyenne 24 points de réussite sur le concept visé, avec un gain individuel allant de 14 points (fonctions, séance 5) à 35 points (boucle *for*, séance 2). Ce chiffre autorise à dire que le canal qualité produit un gain mesurable, et non seulement une variante stylistique du même cours.

Il n'autorise pas à conclure que ce gain se reproduirait à l'identique devant de vrais élèves. Il est mesuré par le même modèle qui a produit à la fois le contenu initial et le contenu réécrit, une réserve qui rejoint le biais d'auto-évaluation discuté en section 9.2.

Le tableau 8 donne, pour les trois mêmes séances, la note produite par le Reviser, la version initiale de la leçon et la version produite en réponse. Ces notes ne sont pas des reformulations vagues du type « être plus clair » : chacune part d'une méprise nommée et demande une modification identifiable. En séance 2, trois élèves sur cinq répondent que *range* ne sait aller que vers le haut, exactement la méprise déclarée du profil *eleve_moyen*. La note demande d'ajouter un exemple à pas négatif avant l'exercice.

En séance 3, la note relève à la fois l'ennui des profils rapides sur un exercice sans risque d'erreur et le refus d'exécution du profil anxieux. Elle propose de faire écrire volontairement une boucle infinie pour la réparer ensuite. Ce niveau de précision tient à la section 4.1 : c'est parce que les méprises y sont nommées plutôt que suggérées que le diagnostic débouche sur une modification identifiable plutôt que sur une consigne générique.

8 Mise en œuvre

Le système repose sur Python 3.11 et deux dépendances d'exécution : le SDK *anthropic* et *python-dotenv*. La suite de tests ajoute une seule dépendance de développement, *pytest*. Aucune base de données n'est utilisée : l'état de la classe et les journaux de run sont des fichiers JSON, lus et réécrits par l'orchestrateur.

Le choix de ne recourir à aucun framework d'agents est justifié en section 3 et n'est pas repris ici.

8.1 Appels structurés plutôt que parsing de texte

Tous les appels au modèle passent par un wrapper unique, *src/llm.py*, fonction *call_structured*. Le principe est le tool use structuré. On déclare un schéma d'entrée (*input_schema*), on force l'appel de l'outil (*tool_choice*), et la fonction retourne un dictionnaire déjà typé, sans étape intermédiaire de sérialisation libre. Le modèle ne produit jamais de texte à interpréter : il remplit un schéma. Aucune extraction de JSON par expression régulière ne traîne dans le code. Cela

Tableau 7: Taux de réussite du concept visé, avant et après le passage du Reviser, sur les trois séances qui ont déclenché une révision.

	Concept	Avant	Après	Gain
S2	boucle for	30%	65%	+35 pts
S3	boucle while	50%	72%	+22 pts
S5	fonctions	45%	59%	+14 pts
Gain moyen				+24 pts

Tableau 8: Les trois séances réécrites. Pour chacune, la note produite par le Reviser à partir du diagnostic, la leçon initiale, et la leçon produite par le Generator en réponse. Les notes ne sont pas des reformulations vagues : chacune part d'une méprise nommée déclarée dans `data/personas/default.json` et demande une modification identifiable.

Note du Reviser	Version initiale	Version produite
Séance 2 — <i>Parcourir dans les deux sens</i> boucle for , 30% → 65%		
Trois élèves sur cinq ont répondu que range ne sait aller que vers le haut : la méprise du profil moyen est confirmée par leurs réponses. Ajouter un exemple à pas négatif avant l'exercice, et leur faire écrire la valeur de i à chaque tour.	La boucle for parcourt une séquence connue à l'avance. for i in range(5) répète cinq fois, avec i qui prend successivement les valeurs 0 à 4.	La boucle for parcourt une séquence connue à l'avance, dans un sens comme dans l'autre. range(5) monte de 0 à 4, range(4, -1, -1) redescend de 4 à 0 : le troisième argument est le pas, et rien n'interdit qu'il soit négatif.
Séance 3 — <i>La boucle qui ne sait pas quand s'arrêter</i> boucle while , 50% → 72%		
Les élèves rapides s'ennuient sur un exercice sans risque d'erreur, et le profil anxieux n'a pas exécuté son code. Faire écrire volontairement une boucle infinie, leur faire prédire ce qui se passe, puis la réparer : l'erreur devient l'objet du cours.	La boucle while répète tant qu'une condition reste vraie. Contrairement au for , elle ne sait pas d'avance combien de tours elle fera.	La boucle while répète tant qu'une condition reste vraie, et c'est là son danger : si rien dans le corps ne rapproche la condition de False , elle tourne indéfiniment. Écrire une boucle qui ne s'arrête pas, puis la réparer, montre mieux qu'un cours ce qui l'arrête.
Séance 5 — <i>Des variables qui ne vivent qu'à l'intérieur</i> fonctions, 45% → 59%		
La réussite sur les <i>variables</i> est retombée à 30% cinq séances après leur introduction. Rattacher les fonctions à un rappel actif de ce concept : faire manipuler des paramètres, qui sont des variables locales, au lieu d'énoncer la règle de portée.	Une fonction met un morceau de code de côté sous un nom, pour le rappeler plus tard sans le réécrire.	Une fonction met un morceau de code de côté sous un nom. Ses paramètres sont des variables qui n'existent qu'à l'intérieur d'elle : les modifier ne change rien dehors. C'est l'étiquette de la première séance, mais collée sur une table qu'on jette en sortant.

supprime d'un coup toute une famille de pannes : réponse tronquée, virgule en trop, guillemets mal fermés. Sur un projet de 48 heures, ces pannes-là sont celles qui coûtent le plus cher, parce qu'elles n'apparaissent pas à la relecture du prompt mais au moment de l'exécution, sur une réponse par ailleurs correcte.

Les prompts système des élèves-agents se répètent à l'identique à chaque itération : seule la mémoire de l'élève change d'un appel à l'autre. Ils sont donc marqués comme cachables (`cache_control: ephemeral`), ce qui évite de retraiter la même portion de prompt à chaque tour de la boucle.

8.2 Mode hors-ligne et suite de tests

`src/fake_agents.py` fournit une implémentation déterministe de chaque agent : mêmes signatures, mêmes types de sortie, mais sans appel au modèle. Cette implémentation s'injecte par paramètre dans `orchestrator.run`, sans modifier l'orchestrateur lui-même. Toute la boucle s'exécute en quelques secondes sans clé API. Un run réel, en appels séquentiels, prendrait de l'ordre de trente minutes. Sur un projet de 48 heures, c'est ce qui sépare un cycle de débogage praticable d'une attente de plusieurs dizaines de minutes à chaque essai.

La suite de tests compte 41 tests, qui mockent tous la couche LLM et passent sans clé API. Une validation humaine s'ajoute à ce dispositif : le plan de séances produit par le Planner est soumis à l'enseignant, qui le confirme ou le refuse, avant toute génération de contenu. Ce contrôle est implémenté comme un hook côté CLI (`main.py`) et non dans l'orchestrateur, pour garder `orchestrator.run()` testable sans `stdin`. Un drapeau `--auto-approve` permet de le sauter.

8.3 Coût du run et effet de la parallélisation

La figure 7 détaille les 60 appels du run de démonstration. Les élèves-agents en concentrent 40 à eux seuls, ce qui correspond à cinq élèves interrogés à chacune des huit itérations. Le Generator et le Diagnostician suivent avec 8 appels chacun, un par itération. Le Reviser compte 3 appels, un par révision, et le Planner un seul appel en tête de run. Les cinq élèves d'une même itération sont indépendants : ils partent ensemble dans un `ThreadPoolExecutor` plutôt que d'attendre leur tour. Le chemin critique du run ne compte alors que 28 attentes successives au lieu de 60 ; la parallélisation en économise 32.

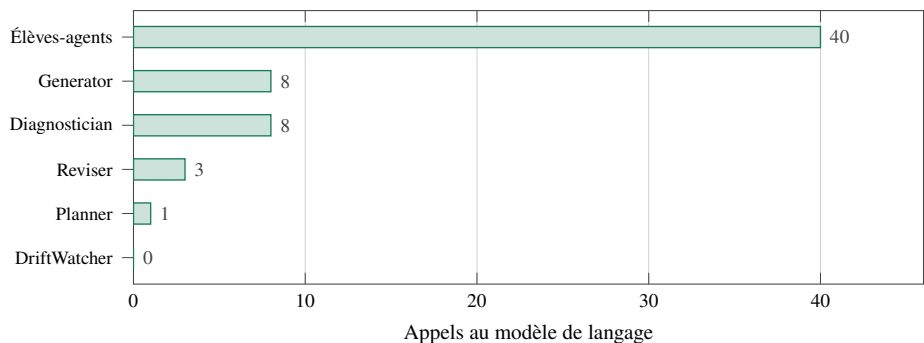


Figure 7: Répartition des 60 appels du run de démonstration. Les élèves-agents en concentrent les deux tiers. Le DriftWatcher, qui valide pourtant chaque réaction d’élève, apparaît à zéro appel : c’est le seul agent entièrement déterministe.

Le contraste que la figure met en évidence tient à la nature du canal plausibilité : ses trois règles déterministes sont du code, pas un jugement délégué au modèle. Le contrôle qui porte sur chaque réaction d’élève ne s’ajoute donc au budget de calcul à aucun endroit du graphique. C’est un effet de la séparation des deux canaux décrite en section 3.2 : faire porter la plausibilité par une couche déterministe libère le budget d’appels pour le canal qualité, où la révision du contenu a, elle, un coût mesurable : 8 appels pour le Diagnostician, 3 pour le Reviser.

9 Discussion et limites

Deux limites tiennent à la nature du système. Elles ne se corrigent pas par un réglage de paramètre ou un prompt mieux écrit. La première porte sur ce que mesurent les élèves-agents ; la seconde, sur qui les juge. Les deux sont énoncées ici avec le protocole qui permettrait de les lever, plutôt que comme des réserves de forme.

9.1 Les élèves-agents ne sont pas des apprenants

Un élève-agent n’est pas un apprenant : c’est un modèle de langage auquel on demande de jouer un rôle, à partir d’une méprise nommée et d’une mémoire longitudinale. Quand ce modèle joue `eleve_difficulte`, il ne reproduit pas le comportement d’un élève réel confondant affectation et égalité. Il reproduit la représentation qu’il se fait, à partir de son corpus d’entraînement, d’un élève qui commet cette confusion. Le système optimise donc son contenu contre un miroir, et rien ne garantit que ce miroir ressemble à une classe réelle.

Ce phénomène est caractérisé sous le nom de caricature dans les simulations par LLM [10] : un modèle chargé de simuler une catégorie de personnes tend à en amplifier les traits stéréotypés plutôt qu’à en reproduire la variabilité réelle. Aher et al. précisent la portée de ce que la simulation de sujets humains par un LLM permet et ne permet pas [9]. Le présent système s’inscrit dans les limites qu’ils posent, sans les dépasser.

Cette limite n’invalide pas la mécanique de mesure. Les taux de réussite du tableau 6 et de la figure 4 sont calculés sur des réponses produites et corrigées, pas sur une impression déclarative. Elle borne en revanche l’interprétation qu’on peut en tirer, puisque la population sur laquelle la mesure porte n’est pas celle qu’elle vise. Un taux de réussite mesuré sur `eleve_difficulte` décrit le comportement d’un modèle jouant une méprise nommée, pas celui d’un élève de lycée confondant = et ==.

9.2 Le biais d’auto-évaluation

Le Generator écrit le contenu d’une séance. Les élèves-agents, en répondant puis en réagissant, produisent le signal à partir duquel le Diagnostician et le Reviser jugent ce même contenu. Les deux rôles s’appuient sur le même modèle. Panickssery et al. montrent que les évaluateurs LLM reconnaissent et favorisent leurs propres productions [12]. Zheng et al. documentent plus largement les biais du LLM-as-a-judge [11] : position, verbosité, style de rédaction. Rien dans l’architecture du système n’empêche ces biais de jouer ici.

La conséquence est concrète : le gain de 24 points en moyenne rapporté en section 7.4 est mesuré par le même modèle qui a produit la réécriture qu’il évalue. Cela interdit de le lire comme une mesure d’efficacité pédagogique. Le DriftWatcher ne corrige pas ce biais. Il contrôle la plausibilité de la simulation, c’est-à-dire si la trajectoire de mémoire d’un élève-agent respecte les règles R1–R3, et non la complaisance du jugement porté sur le contenu, qui reste dans le canal qualité.

9.3 Protocole de validation proposé

Le protocole qui permettrait de mesurer l’ampleur des deux limites précédentes est simple à énoncer, et n’a pas été mené dans les 48 heures du projet. Il consiste à rejouer un sous-ensemble des séances générées avec de vrais élèves, à comparer leur taux de réussite réel par concept à celui prédit par le Diagnostician pour les mêmes séances, et à mesurer l’écart entre les deux courbes. La métrique s’y prête bien, parce qu’elle est définie sur des réponses corrigées et non sur une impression

déclarative : la comparaison porte sur deux taux de réussite de même nature, pas sur un taux mesuré face à une opinion recueillie par questionnaire.

Tant que cet écart n'est pas mesuré, les courbes de ce rapport décrivent le comportement du simulateur, pas celui d'une classe. Cette conclusion n'est pas atténuée par la qualité de l'architecture, ni par le nombre de contrôles déterministes qu'elle embarque. Un système qui mesure juste sur une population qu'il a lui-même simulée reste, jusqu'à cette validation, un système qui se mesure lui-même.

10 Travaux futurs

Trois directions se dégagent de ce qui existe déjà. La première prolonge le Generator en planificateur de rappels. Aujourd'hui, il ajoute des exercices de reprise quand le Reviser le lui suggère à la suite d'un diagnostic, mais rien ne l'empêcherait de cibler, à chaque itération, les concepts dont la demi-vie mesurée est la plus courte. Boucler ainsi la métrique de la section 6.2 sur elle-même, en faire une entrée du système de génération et non plus seulement une sortie du rapport, rejoint les travaux sur la planification des révisions espacées [3, 4].

La deuxième direction répond à la limite posée en section 9.2 : séparer les modèles, un pour écrire le contenu, un autre pour jouer les élèves-agents qui le jugent. Le changement ne touche à aucune autre partie de l'architecture ; les deux canaux, les six rôles et les trois règles déterministes du DriftWatcher restent identiques. Il coupe seulement la possibilité qu'un modèle favorise ses propres productions.

La troisième est la validation contre de vrais élèves décrite en section 9.3 : rejouer un sous-ensemble des séances avec une classe réelle et mesurer l'écart entre le taux de réussite prédit et le taux de réussite observé. C'est la direction la plus coûteuse des trois à mettre en œuvre. C'est aussi la seule qui permettrait de faire passer les chiffres de ce rapport du statut d'illustration à celui de résultat.

11 Conclusion

Le système ne note pas un cours sur l'effet qu'il produit à l'instant où il est lu. Il le note sur ce qu'il en reste plusieurs séances plus tard, il le mesure sur des réponses corrigées, et il garde ses propres élèves synthétiques sous contrôle déterministe pour que cette mesure veuille dire quelque chose. Cette dernière précaution est ce qui distingue le système du piège qu'il aurait pu ne pas éviter : réécrire un cours parce qu'un élève simulé a déraillé, plutôt que parce que le contenu le méritait.

Ce que le projet met en place tient en quelques éléments concrets. Six rôles se répartissent sur deux canaux de signal disjoints, et trois règles déterministes filtrent chaque mutation de mémoire. Une métrique, le taux de réussite par concept, en fournit une demi-vie mesurée séance après séance. Le run de démonstration compte 60 appels au modèle, dont la parallélisation ramène le chemin critique à 28 attentes successives.

Les chiffres présentés dans ce rapport restent, à ce stade, une illustration du fonctionnement du système et non un résultat expérimental. Ils proviennent d'un unique run construit pour exercer tous les cas de figure à la fois, pas d'une série de runs indépendants ni d'une comparaison à des élèves réels. Changer ce statut suppose le protocole décrit en section 9.3 : rejouer les séances générées avec de vrais élèves et mesurer l'écart. C'est la condition à laquelle ce système cesserait de se mesurer lui-même.

Références

- [1] H. Ebbinghaus, "Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie," Duncker & Humblot, Leipzig, 1885. Traduction anglaise de référence : H. A. Ruger, C. E. Bussenius (trad.), "Memory: A Contribution to Experimental Psychology," Teachers College, Columbia University, New York, 1913.
- [2] H. L. Roediger, J. D. Karpicke, "Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention," *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.
- [3] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, D. Rohrer, "Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis," *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.
- [4] B. Settles, B. Meeder, "A Trainable Spaced Repetition Model for Language Learning," *Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1848–1858, 2016.
- [5] A. T. Corbett, J. R. Anderson, "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [6] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, J. Sohl-Dickstein, "Deep Knowledge Tracing," *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 505–513, 2015.

- [7] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” *ACM Symposium on User Interface Software and Technology (UIST)*, pp. 1–22, 2023.
- [8] J. M. Markel, S. G. Opferman, J. A. Landay, C. Piech, “GPTeach: Interactive TA Training with GPT-based Students,” *ACM Conference on Learning @ Scale (L@S)*, pp. 226–236, 2023.
- [9] G. V. Aher, R. I. Arriaga, A. T. Kalai, “Using Large Language Models to Simulate Multiple Humans and Replicate Human Subject Studies,” *International Conference on Machine Learning (ICML)*, pp. 337–371, 2023.
- [10] M. Cheng, T. Piccardi, D. Yang, “CoMPosT: Characterizing and Evaluating Caricature in LLM Simulations,” *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 10853–10875, 2023.
- [11] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, pp. 46595–46623, 2023.
- [12] A. Panickssery, S. R. Bowman, S. Feng, “LLM Evaluators Recognize and Favor Their Own Generations,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 68772–68802, 2024.
- [13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 8634–8652, 2023.
- [14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 46534–46594, 2023.
- [15] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, “Large Language Models Cannot Self-Correct Reasoning Yet,” *International Conference on Learning Representations (ICLR)*, 2024.